

```
teacher_information=fields.EmbeddedField(teacherField)
semester_1_subjects=fields.EmbeddedField(sem1Field)
semester_2_subjects=fields.EmbeddedField(sem2Field)
```

We are treating the distinct fields within 'detailed_data', notably 'teacher_information' (a dictionary), 'semester_1_subjects' and 'semester_2_subjects' (both are lists of dictionaries), as separate JSON models and then giving a call to them using the embedded field within the outlying 'detailed_data_field' JSON model. The 'EmbeddedField' mentioned in the last three lines above performs the same function as that of foreign keys in databases.

Updated views.py

```
from marksapp1.models import detailed_data_field ----(1)
class StudentView(APIView):
    def post(self,request):
        data=request.data
        serializer=StudentSerializer(data=request.data)
----(5)
        if serializer.is_valid():
            data1=detailed_data_field()
            myschema=data1.to_json_schema()
            print(myschema)
            v=Draft7Validator(myschema) ----
(10)
            if len(list(v.iter_errors(data["detailed_
data"])))!=0:
                return Response({"error":str(list(v.iter_
errors(data["detailed_data"])))})
            else:
                serializer.save()
                return Response(serializer.
data,status=status.HTTP_201_CREATED) ----(15)
            else:
                return Response(serializer.errors,status=status.
HTTP_400_BAD_REQUEST)
```

In line (7), our overall JSON model 'detailed_data_field' is assigned to variable 'data1', followed by schema generation of the same in line (8). A glimpse of the schema generated is seen below.

```
{ ----(1)
'type': 'object',
'additionalProperties': False,
'properties': {
    'date_of_joining': {'type': 'string'},
----(5)
    'father_name': {'type': 'string'},
```

```
    'mother_name': {'type': 'string'},
    'semester_1_subjects': {'type': 'object',
        'additionalProperties': False,
        'properties': {'professor': {'type':
'string'}, ----(10)
            'score': {'type': 'number'},
            'subject_name': {'type': 'string'}
        },
        'required': ['professor', 'score', 'subject_
name']
    }, ----(15)
    'semester_2_subjects': {'type': 'object',
        'additionalProperties': False,
        'properties': {'professor': {'type':
'string'},
            'score': {'type': 'number'},
            'subject_name': {'type': 'string'}
        }, ----(20)
        'required': ['professor', 'score', 'subject_
name']
    },
    'teacher_information': {'type': 'object',
        'additionalProperties': False,
----(25)
        'properties': {'date_of_birth': {'type':
'string'},
            'department': {'type': 'string'},
            'joining_date': {'type': 'string'},
            'name': {'type': 'string'}
        }, ----(30)
        'required': ['date_of_birth', 'department',
'joining_date', 'name']
    },
    'required': ['date_of_joining', 'father_name', 'mother_
name']}
```

We can observe in the auto-generated schema that for 'semester_1_subjects' and 'semester_2_subjects', the 'type' is 'object', whereas it should actually be 'array'. Among the fields that exist for JSON models, there are none that help develop a *type:'array'* in the schema. To visualise this, let us compare the 'semester_1_subjects' structure from the auto-generated schema with the manually written one.

Auto-generated schema

```
'semester_1_subjects': {'type': 'object',
    'additionalProperties': False,
    'properties': {'professor': {'type':
```